AccessWatch Group checklist: what to do, where, and how

SANGEETHA — Platform Administrator

Check off each of these yourself against `AdminController.cs` and its views:

1. Manage user accounts

- **Where:** `ManageUsers.cshtml` + `CreateUser`/`DeleteUser` actions
- **What it should do:** list all users, create new accounts for Admin/Inspector/Maintenance roles, delete accounts (blocked if it's the last remaining admin)

2. Review accessibility reports

- **Where:** `ReviewReports.cshtml` / `ReviewReports` action
- **What it should do:** list all reports with status "Submitted," awaiting a decision

3. Approve or reject reports

- **Where:** `ApproveAndAssign` and `RejectReport` actions
- **What it should do:** approving records who reviewed it and moves status forward; rejecting records the decision and stops the report there

4. Assign reports to inspectors

- **Where:** part of `ApproveAndAssign` (the inspector dropdown on `ReviewReports.cshtml`)
- **What it should do:** picks an Accessibility Inspector account and links them to the report, changes status to "Assigned"

5. Manage categories

- **Where:** `ManageCategories.cshtml` + `CreateCategory`/`DeleteCategory` actions
- **What it should do:** add and delete accessibility categories (e.g. "Ramp," "Lift")

6. Monitor system activities

- **Where:** `Index.cshtml` (dashboard)
- **What it should do:** shows total users, total reports, and counts broken down by status, plus a recent activity table

Optional, not required: an Edit function for categories (currently only add/delete), or a chart instead of plain numbers on the dashboard.

ABDULLAH — User

What's already working: Register, Login, Submit report, My Reports. (understand how it works so bcs you have to explain it in the video)

Still needs:

1. Update Profile

- **Where:** add to `ViewModels/AccountViewModels.cs`, `Controllers/AccountController.cs`, and a new `Views/Account/UpdateProfile.cshtml`
- **What to do:** build a form where the logged-in user can change their Name and Email. Load their current info when the page opens (GET), save the changes when they submit (POST). Check that a new email isn't already taken by someone else before saving. After saving, the user needs to be signed in again with fresh info so the nav bar greeting updates.
- **How:** follow the same pattern as `Register` (same controller, same file) — a GET action that shows a form, a POST action that validates and saves.

2. View Updates (in the report details page)

- **Where:** `Controllers/ReportController.cs` (the `Details` action) and `Views/Report/Details.cshtml`
- **What to do:** the report details page currently only shows the report itself. It needs to also pull in and display the related Inspection record (findings, rating, recommended action) and Repair record (progress, corrective actions, completion photo) once those exist.
- **How:** when fetching the report in the controller, also load its related Inspection and Repair records, not just the report's own fields. In the view, add a section that only displays if an Inspection exists, and another that only displays if a Repair exists.

- **Dependency:** this can only be fully tested once Qusai and Omer's controllers exist and actually create Inspection/Repair records.

OMER — Accessibility Inspector

Nothing built yet. 6 functionalities to cover: view assigned cases, update inspection status, submit findings, provide rating, recommend action, forward issue.

1. View assigned cases

- **Where:** new `Controllers/InspectionController.cs`, new `Views/Inspection/Index.cshtml`
- **What to do:** show a list of reports that are assigned to the logged-in inspector and still waiting on inspection. Each row should show who submitted it and when, with a link into the findings form.

2. Update inspection status

- **Where:** same controller, either its own small action or a status control added to `Views/Inspection/Index.cshtml`
- **What to do:** let the inspector mark a case as "in progress" once they start working on it — a distinct step from submitting final findings, so it's not just an automatic side effect of the findings form. Without this, "update inspection status" isn't really its own functionality, it's just folded into step 3.

3. Submit findings, rating, recommend action, forward issue

- **Where:** same controller, new `Views/Inspection/Findings.cshtml`
- **What to do:** one form covering all four functionalities together — findings text, a 1–5 rating, a recommended action, and a checkbox for "forward to maintenance." On submit: create a new Inspection record linked to that report, and update the report's status (to "in repair" if forwarded, otherwise "inspected"). Check the report is actually assigned to this inspector before allowing the update.

4. View inspection history

- **Where:** same controller, new `Views/Inspection/History.cshtml`
- **What to do:** list of the inspector's own past inspections — what report, when, what rating/findings were given.

5. Nav link

- **Where:** `Views/Shared/_Layout.cshtml`
- **What to do:** add an "Assigned Cases" link that only shows for users with the Accessibility Inspector role.

QUSAI — Facility Maintenance Officer

Nothing built yet. Can build the structure now, but full testing needs Qusai to forward a report first.

1. View repair tasks + review inspection recommendations

- **Where:** new `Controllers/FacilityController.cs`, new `Views/Facility/Index.cshtml`
- **What to do:** list reports that have been forwarded for repair. For each one, show the inspector's recommended action alongside it (this covers "review recommendations" — it's the same page, not a separate one) and a link into the update form.

2. Update progress, record actions, upload evidence, mark completed

- **Where:** same controller, new `Views/Facility/UpdateRepair.cshtml`
- **What to do:** one form covering all four — a progress selector, a corrective actions text box, a photo upload for completion evidence, and a "mark completed" checkbox. On submit: create or update a Repair record linked to the report. If marked completed, also flip the parent report's status to completed.

3. Nav link

- **Where:** `Views/Shared/_Layout.cshtml`
- **What to do:** add a "Repair Tasks" link that only shows for users with the Facility Maintenance Officer role.

Suggested order

1. Sangeetha — nothing left, help others if needed
2. Abdullah — Update Profile first (independent, testable now)
3. Qusai — build now, real assigned report already exists to test against

4. Omer — build structure now, full test waits on Qusai forwarding something
5. Everyone together — full loop test: submit → review & assign → inspect & forward → repair & complete → confirm updates show on Abdullah's details page